

Applications of Neural Networks in Continuous and Dynamic Systems

CSCI 5922 — Fundamentals of Neural Networks and Deep Learning

Nick Mobus & Sean Fitzgerald

29 April 2026

Abstract

Many real-world systems exhibit non-stationary behavior, making them difficult to model using traditional statistical approaches that assume fixed underlying distributions. In this work, we investigate the ability of neural network architectures to recover the latent dynamics of stochastic, time-varying processes. We construct two synthetic non-stationary datasets based on Bernoulli processes with evolving probability parameters, incorporating both deterministic and probabilistic regime-switching behavior, as well as additive Gaussian noise. We evaluate six models, an auto-regressive multilayer perceptron (MLP), a recurrent neural network (RNN), a long short-term memory network (LSTM), a Transformer, a Neural Ordinary Differential Equation (ODE), and a Neural Jump ODE, on their ability to estimate the underlying probability at each time step. Model performance is assessed using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and correlation between predicted and true probabilities. Our results show that recurrent architectures, particularly LSTMs, outperform simpler auto-regressive models, while continuous-time approaches further improve performance. The Neural Jump ODE consistently achieves the strongest results across evaluation metrics, highlighting the importance of explicitly modeling regime changes and discontinuities in non-stationary environments. These findings suggest that architectures designed to capture temporal dynamics and structural changes are best suited for learning complex, time-varying stochastic processes.

1 Introduction

Modern deep neural networks have achieved startling results in their ability to capture knowledge about the systems they model. There are, however, limitations to this ability. Once training has occurred the learned parameters are fixed, ossifying the current knowledge state of the model. This is incompatible with many systems that we desire to model or predict in. These systems tend towards dynamic behaviors where the future states of the system are dependent not just on the underlying parameters which define the system, but the current state of the system as well. As such, neural networks as we currently use them exhibit difficulty effectively modeling and predicting these systems. We believe that the next major advancement in deep learning will involve improvements in model architectures and data structuring that addresses these areas of concern and allow for better

modeling of dynamic systems.

Neural networks are powerful and capable of learning nuanced understanding of the underlying properties of significantly complex systems; you need look no further than the massive strides in generative language modeling accomplished by modern Large Language Models for evidence. But in order to keep these models current and up-to-date, modern LLMs require relatively frequent re-training. This process is incredibly expensive, often costing millions of dollars, and extraordinarily time consuming and computationally expensive, taking weeks or even months. The process of accumulating the training dataset is likewise expensive and laborious, and combining these factors means that these cutting edge models often have a knowledge cutoff between 6 to 18 months prior to their release. Since 2015, there has been significant research into approaches and differing model architectures that address this problem. Techniques include continual/lifelong learning, adaptive learning, and adaptive computation. Architectures include liquid neural networks, neural ordinary differential equations (ODEs), and neural jump ODEs. The research on these approaches show strong promise. However, we observed throughout our literature review that most research focused on evaluating these approaches on predictive accuracy. Although this evaluation method is important for showing how well an approach performs in a certain environment, it does not show if the model is truly understanding the underlying dynamic nature of the system.

Our research will focus on testing whether different neural architectures can recover the underlying stochastic dynamics of a non-stationary system. We will create two synthetic dynamic datasets, which are binary and governed by stochastic elements. We will then train each tested architecture on the given dataset. However, our approach will not evaluate how well each model predicts the binary outcome, but instead will examine how well each model predicts the underlying stochastic probability of the system at that point. This will allow us to evaluate whether the model is able to recover the underlying dynamics and in turn prove an effective method for model selection for dynamic systems.

2 Related Work

Modeling time series data using neural networks has been widely studied, with a large body of work focusing on forecasting and representation learning for sequential data. Early and foundational efforts have explored the use of feedforward and recurrent neural networks for time series prediction, often within autoregressive frameworks. A systematic review by Tealab Tealab 2018 highlights that while neural networks have been extensively applied to forecasting tasks, relatively few works propose fundamentally new modeling paradigms beyond autoregressive structures. This observation motivates the exploration of alternative architectures capable of capturing more complex and non-stationary dynamics.

More recent surveys have expanded this perspective by examining a broader range of neural architectures for time series modeling. Muñoz-Zavala et al. Muñoz-Zavala et al. 2024 review the application of feedforward, radial basis function, and recurrent neural networks, emphasizing both their strengths and limitations in capturing temporal dependencies. Their analysis underscores that while these models are effective in many settings, challenges remain in modeling non-stationary processes and adapting to evolving statistical properties over time.

A complementary line of research focuses on dynamic neural networks, which adapt their structure or parameters based on input data. Han et al. Han et al. 2022 provide a comprehensive survey of

such models, including temporal-wise dynamic architectures that modify their behavior across time. These approaches highlight the importance of adaptivity in sequential modeling, particularly in non-stationary environments where fixed-structure models may be insufficient.

Beyond traditional sequence modeling, neural network architectures have also been developed for decision-making in dynamic environments. For example, Villacorta-Atienza and Makarov Villacorta-Atienza and Makarov 2013 propose a neural architecture for cognitive navigation that incorporates internal representations of time-evolving environments. Their framework combines learned representations with adaptive decision-making pathways, demonstrating robustness to uncertainty and changing conditions. Although developed in the context of robotic navigation, this work illustrates the broader importance of modeling evolving latent states in dynamic systems.

Despite these advances, much of the existing literature focuses either on forecasting performance or on specific application domains, rather than on systematically evaluating how different neural architectures capture underlying latent dynamics. In particular, there is limited work comparing feedforward, recurrent, attention-based, and continuous-time models within a controlled setting where the true data-generating process is known. In this work, we address this gap by constructing synthetic non-stationary Bernoulli processes with regime-switching behavior and evaluating a diverse set of neural architectures on their ability to recover the latent probability. This approach enables a direct comparison of model inductive biases and their effectiveness in capturing non-stationary temporal structure.

3 Methods

3.1 Synthetic Data Generation

We constructed synthetic non-stationary Bernoulli sequences in which the latent success probability p_t evolves as a function of past observations. At each timestep, a binary outcome $x_t \in \{-1, +1\}$ is sampled according to $\Pr(x_t = 1) = p_t$. While the generative process is defined using $\{-1, +1\}$, targets are mapped to $\{0, 1\}$ during training to align with the binary cross-entropy loss.

Rather than modeling p_t directly, the process evolves in a latent logit space. A latent variable $z_t \in \mathbb{R}$ is transformed into a probability via a scaled sigmoid function,

$$p_t = \ell + (u - \ell) \sigma(z_t),$$

where $\sigma(\cdot)$ is the sigmoid function and $[\ell, u] = [0.25, 0.75]$ constrains the range of p_t .

The latent state evolves according to two regimes. In the trend regime, the latent state is incremented in the direction of the current observation, encouraging persistent upward or downward movement in p_t . In the mean-reverting regime, the latent state is contracted toward zero before incorporating the current observation, producing stabilizing dynamics.

We generated two variants of this process. In the deterministic setting, regime transitions are governed by threshold rules on p_t , with switches to the mean-reverting regime occurring at extreme values and switches back to the trend regime occurring near the center. This produces structured, piecewise dynamics with predictable switching behavior. In the stochastic setting, regime transitions occur probabilistically, with switching probability increasing as p_t approaches extreme values and decreasing near $p_t = 0.5$. Gaussian noise is additionally injected into the latent dynamics at each timestep, resulting in smoother and less predictable trajectories.

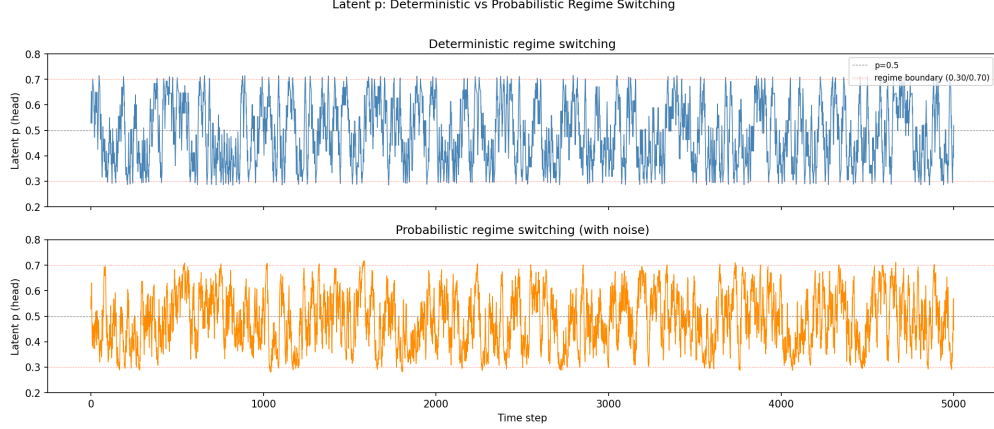


Figure 1: True P for Deterministic and Probabilistic Regime Change Data

3.2 Dataset Construction

From each simulated sequence, we constructed supervised learning datasets tailored to different model classes. For autoregressive models, fixed-length windows of past observations were used as inputs to predict the next outcome. For sequence models, full sequences were used, where the input consists of all observations up to time $t - 1$, and the target is the next observation. The latent probability p_t is retained for evaluation but is not used for supervision.

3.3 Model Architectures

We evaluated a range of neural architectures designed to capture temporal dependencies and non-stationary dynamics. All models take as input sequences of past binary observations and produce a probability estimate for the next outcome. Most models output scalar logits which are passed through a sigmoid function, while the autoregressive MLP outputs probabilities directly.

Autoregressive MLP. The autoregressive MLP is a feedforward baseline operating on fixed-length windows. Given an input vector $x_{t-k:t-1} \in \mathbb{R}^k$, consisting of the previous k outcomes, the model predicts the probability of the next outcome. The network consists of two hidden layers of size 64 with LeakyReLU activations, followed by a sigmoid output layer:

$$\hat{p}_t = \sigma(f_\theta(x_{t-k:t-1})).$$

This model does not maintain a recurrent state and relies entirely on the input window.

RNN. The vanilla RNN processes the full sequence and produces predictions at each timestep. The hidden state evolves according to

$$h_t = \tanh(W_x x_t + W_h h_{t-1}),$$

where $h_t \in \mathbb{R}^{32}$. A linear output layer maps each hidden state to a scalar logit:

$$\hat{y}_t = W_{\text{out}} h_t.$$

LSTM. The LSTM extends the RNN with gating mechanisms that regulate information flow. At each timestep,

$$(h_t, c_t) = \text{LSTM}(x_t, h_{t-1}, c_{t-1}),$$

where $h_t, c_t \in \mathbb{R}^{64}$. A linear layer produces logits at each timestep.

Transformer. The Transformer is a causal sequence model with learned input and positional embeddings. Inputs are projected to a $d_{\text{model}} = 128$ dimensional space:

$$h_t^{(0)} = W_{\text{in}} x_t.$$

A learned positional embedding is added, and the sequence is processed by a stack of three Transformer encoder layers with 8 attention heads and feedforward dimension 256. A strict causal mask ensures that each timestep attends only to past and current inputs. The final representations are normalized and mapped to logits via a linear layer.

Neural ODE. The Neural ODE model augments a recurrent update with continuous-time latent evolution. At each timestep,

$$h_t^{\text{disc}} = \tanh(W_x x_t + W_h h_{t-1}),$$

followed by continuous evolution governed by

$$\frac{dh(t)}{dt} = f_{\theta}(h(t)),$$

where f_{θ} is a two-layer neural network with Tanh activation. The ODE is integrated over $t \in [0, 1]$ using a fixed-step RK4 solver. The result is combined with the discrete update via a residual connection:

$$h_t = \text{LayerNorm}(h_t^{\text{disc}} + \text{ODE}(h_t^{\text{disc}})).$$

Neural Jump ODE. The Neural Jump ODE separates continuous evolution from discrete updates. The hidden state is initialized from the first observation, then evolves according to the same ODE between timesteps. At each timestep, a jump update is applied:

$$h_t = h_t^{\text{cont}} + g_{\phi}(h_t^{\text{cont}}, x_t),$$

where g_{ϕ} is a two-layer neural network operating on the concatenation of h_t^{cont} and x_t . The result is normalized and used for prediction.

3.4 Training Procedure

All models were trained to predict the next binary outcome using binary cross-entropy loss. For models that output logits, this was implemented as `BCEWithLogitsLoss`, while the autoregressive MLP was trained using `BCELoss`. Optimization was performed using Adam or AdamW depending on the model, with model-specific learning rates and regularization settings as described above. The best-performing model was selected based on validation loss.

4 Experiments

Main Purpose The purpose of our experiment was to evaluate how effectively each of the six models could recover the underlying stochastic dynamics of two synthetic non-stationary datasets. As discussed in the Methods section, each dataset was generated from a Bernoulli process with a time-varying probability parameter. For each model, the final sigmoid output was interpreted as an estimate of the latent probability at each time step. Model performance was then assessed by comparing these predicted probabilities to the true underlying probabilities, enabling a direct evaluation of each model’s ability to recover the temporal structure of the underlying stochastic process, rather than simply predicting observed outcomes.

Evaluation Metric(s) We evaluated model performance with three metrics: Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the correlation between predicted and true probability values. MAE provides a direct measure of the average magnitude of prediction error, offering an interpretable assessment of how closely each model’s estimates match the true latent probabilities. RMSE places greater emphasis on larger errors, making it particularly useful for identifying models that occasionally produce significant deviations from the underlying process. This metric helps identify models that struggle at identifying the points of regime change within the datasets. Finally, correlation measures the strength of the linear relationship between predicted and true probabilities, capturing how well each model recovers the temporal structure and regime trends of the underlying stochastic dynamics, independent of scale. Together, these metrics provide a comprehensive evaluation of both point-wise accuracy and the ability to track the evolving behavior of the data-generating process.

Results Figure 2 below shows a model to model comparison for each evaluation metric MAE (Figure 2a), RMSE (Figure 2b), and Correlation (Figure 2c). Each model has two bars, a blue bar and an orange bar. The blue bars represent the results of the experiment utilizing the clean dataset while the orange bars represent the results of the experiment utilizing the noisy dataset.

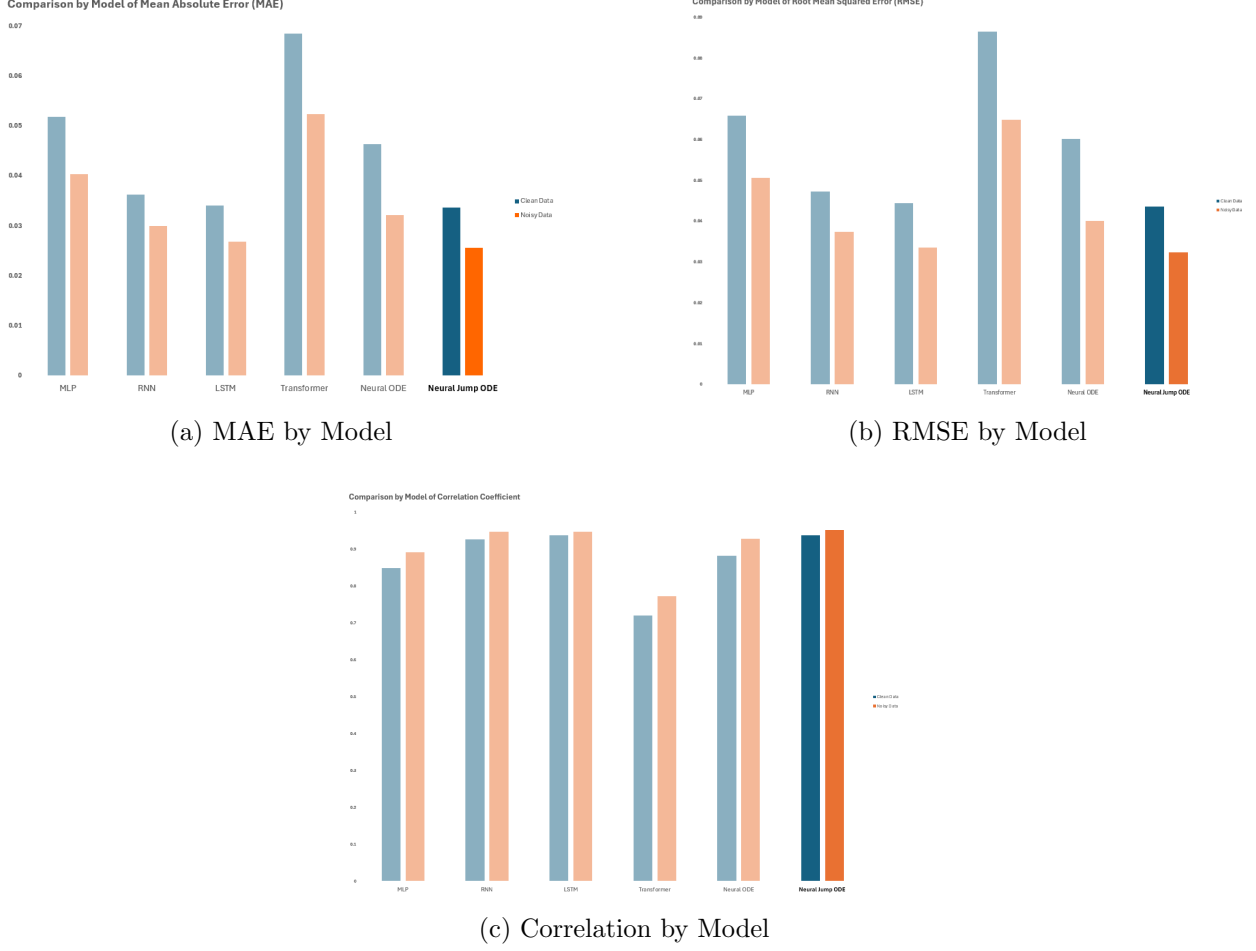


Figure 2: Experiment Results

From the above figure, it can be seen that the Neural Jump ODE achieved the strongest results across all three evaluation metrics.

Table 1 below summarizes the results shown in the figure above.

Table 1: Model Performance on Clean and Noisy Datasets

Model	Clean Data			Noisy Data		
	MAE	RMSE	Corr	MAE	RMSE	Corr
MLP	0.0518	0.0658	0.8493	0.0404	0.0506	0.8911
RNN	0.0362	0.0472	0.9266	0.0300	0.0374	0.9467
LSTM	0.0340	0.0444	0.9373	0.0268	0.0335	0.9473
Transformer	0.0685	0.0864	0.7207	0.0524	0.0648	0.7725
Neural ODE	0.0464	0.0601	0.8827	0.0321	0.0401	0.9291
Neural Jump ODE	0.0336	0.0435	0.9374	0.0256	0.0324	0.9521

As shown in Table 1, the Neural Jump ODE consistently achieves the best performance across all

evaluation metrics, with the LSTM emerging as the second-best model.

Figure 3 and Figure 4 below show the results from the Neural Jump ODE model. The orange line is the predicted probability while the blue line is the true underlying probability.

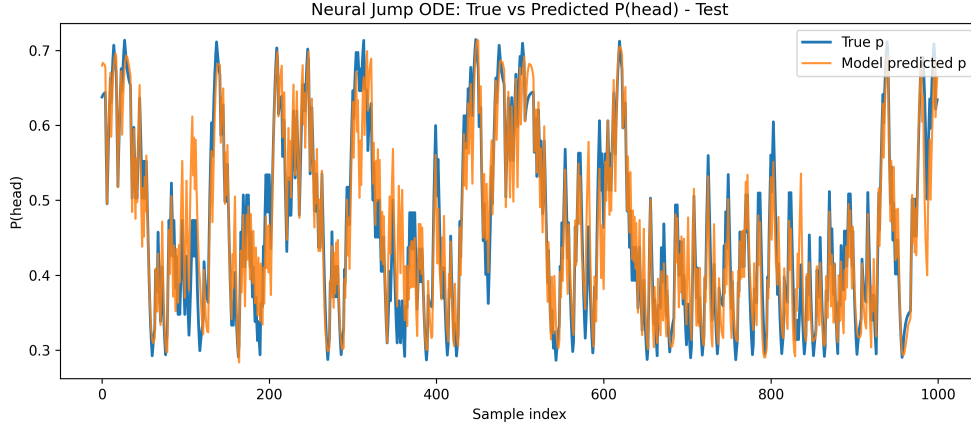


Figure 3: Simple Model Performance

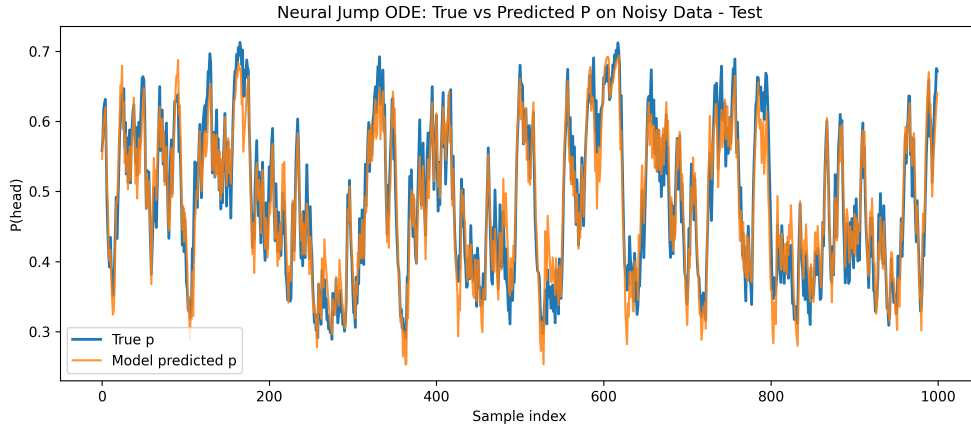


Figure 4: Simple Model Performance

These figures demonstrate the surprising ability of the Neural Jump ODE model to recover the underlying stochastic dynamics and shows promise for the use neural network based architectures in dynamic and non-stationary systems.

One interesting trend from our experiment was that every model tested performed better on the noisy dataset as compared to the clean dataset. While this may initially seem counterintuitive, a likely explanation lies in the nature of the regime switching dynamics. The clean dataset follows a strictly deterministic switching process, leading to sharp transitions in the underlying probability that can be difficult for models to anticipate and accurately track. In contrast, the noisy dataset incorporates probabilistic transitions and additive Gaussian noise, resulting in smoother and more gradual changes in the latent probability. This increased continuity may make the underlying dynamics easier for models to learn, leading to improved performance across evaluation metrics.

The results suggest that, in this setting, the predictability and smoothness of the underlying process play a more significant role in model performance than the presence of observational noise.

While our results demonstrate clear performance differences across different model, several important questions remain unresolved. First, our experiments are limited to synthetic Bernoulli processes with controlled regime-switching dynamics. While this was a great first step, this data most likely does not fully capture the complexity of real world non-stationary systems. It is therefore unclear how well these findings generalize to more realistic settings with higher-dimensional inputs, longer temporal dependencies, or partially observed states. Additionally, while the Neural Jump ODE achieved the strongest overall performance, our study does not isolate which components of the architecture contribute most significantly to this result. Future work could address these limitations by evaluating models on real-world datasets, systematically varying the degree and type of non-stationarity, and conducting ablation studies to better understand the mechanisms driving performance gains.

5 Conclusions

The primary contribution of this work is a controlled comparison of diverse neural architectures on their ability to model non-stationary stochastic processes and recover underlying latent dynamics. By constructing synthetic Bernoulli sequences with regime-switching behavior and known latent probabilities, we created a setting in which model performance can be evaluated not only in terms of predictive accuracy but also in terms of latent state reconstruction. Across autoregressive, recurrent, attention-based, and continuous-time models, our results highlight that architectural inductive biases play a critical role in capturing non-stationarity. In particular, models that incorporate mechanisms for temporal memory, adaptive updates, or continuous latent evolution demonstrate improved ability to track the evolving probability process, while simpler feedforward approaches are limited by their fixed receptive fields; additionally, transformer based models may under-perform expectations without significant hyperparameter tuning. These findings suggest that explicitly or implicitly modeling temporal structure is essential for learning in non-stationary environments.

From an ethical perspective, while this project is based on synthetic data and does not directly involve sensitive real-world applications, the broader class of models studied here is frequently applied to domains such as finance, healthcare, and behavioral prediction, where non-stationarity is pervasive and decisions can have significant consequences. In such settings, errors in modeling changing dynamics may lead to biased or unreliable predictions, particularly if models fail to adapt to shifts in underlying distributions. This raises concerns related to fairness, accountability, and robustness, especially when models are deployed in dynamic environments without appropriate monitoring or recalibration. Additionally, more complex models such as Transformers and Neural ODEs may reduce transparency and interpretability, making it difficult to understand how predictions are formed or why failures occur. These considerations underscore the importance of developing methods that are not only accurate but also interpretable, adaptive, and robust to distributional change, particularly when applied to real-world decision-making systems.

6 Bibliography

References

- Han, Yizeng et al. (Nov. 2022). “Dynamic Neural Networks: A Survey”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44.11, pp. 7436–7456. ISSN: 1939-3539. DOI: 10.1109/TPAMI.2021.3117837.
- Muñoz-Zavala, Angel E. et al. (2024). “A Literature Review on Some Trends in Artificial Neural Networks for Modeling and Simulation with Time Series”. In: *Algorithms* 17.2. ISSN: 1999-4893. DOI: 10.3390/a17020076. URL: <https://www.mdpi.com/1999-4893/17/2/76>.
- Tealab, Ahmed (2018). “Time series forecasting using artificial neural networks methodologies: A systematic review”. In: *Future Computing and Informatics Journal* 3.2, pp. 334–340. ISSN: 2314-7288. DOI: <https://doi.org/10.1016/j.fcij.2018.10.003>. URL: <https://www.sciencedirect.com/science/article/pii/S2314728817300715>.
- Villacorta-Atienza, José Antonio and Valeri A. Makarov (Dec. 2013). “Neural Network Architecture for Cognitive Navigation in Dynamic Environments”. In: *IEEE Transactions on Neural Networks and Learning Systems* 24.12, pp. 2075–2087. ISSN: 2162-2388. DOI: 10.1109/TNNLS.2013.2271645.